



JWT e OAuth2

Programação Web II

Cenários de Integração com Múltiplas Aplicações

Desafios de integração

Em ambientes corporativos, aplicações web frequentemente interagem com vários serviços e APIs.

Garantir uma autenticação segura e centralizada torna-se essencial para evitar múltiplos logins e fornecer uma experiência de usuário unificada.

Cenários de Integração com Múltiplas Aplicações

Soluções de autenticação centralizada

Single Sign-On (SSO): permite que um usuário faça login uma única vez e tenha acesso a diferentes aplicações conectadas, sem a necessidade de múltiplas autenticações.

OAuth2: um protocolo de autorização amplamente utilizado para permitir que aplicações acessem recursos protegidos em nome de um usuário.

Autenticação com JWT (JSON Web Token)

Programação Web II

O que é JWT?

O *JSON Web Token* (JWT) é um padrão aberto para transmissão de informações de forma segura entre duas partes.

Ele é compactado e assinado, permitindo verificação de integridade e autenticação sem estado.

Leia mais: [JWT](#)

Autenticação com JWT (JSON Web Token)

Programação Web II

Estrutura do JWT

- **Header:** Contém o tipo de token e o algoritmo de assinatura.
- **Payload:** Contém declarações (*claims*) como identificadores e permissões.
- **Signature:** Verifica a autenticidade e a integridade do token.

Exemplo de JWT:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0Ij0iYWRtaW4iOnRydWUsImVudCI6MTUxNjIzOTAyMn0.KMUFsIDTnFmyG3nMiGM6H9FNFUR0f3wh7SmqJp-QV30
```

Autenticação com JWT (JSON Web Token)

Programação Web II

Configurando JWT no Spring Security:

pom.xml

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-api</artifactId>
  <version>0.11.2</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-impl</artifactId>
  <version>0.11.2</version>
</dependency>
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt-jackson</artifactId>
  <version>0.11.2</version>
</dependency>
```

Autenticação com JWT (JSON Web Token)

Programação Web II

Configurando JWT no Spring Security:

build.gradle

```
dependencies {  
    implementation 'org.springframework.boot:spring-boot-starter-security'  
    implementation 'io.jsonwebtoken:jjwt-api:0.11.2'  
    runtimeOnly 'io.jsonwebtoken:jjwt-impl:0.11.2'  
    runtimeOnly 'io.jsonwebtoken:jjwt-jackson:0.11.2'  
}
```


Autenticação com JWT (JSON Web Token)

Programação Web II

Configurando JWT no Spring Security:

```
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import java.util.Date;

public class JwtUtil {

    private static final String SECRET_KEY = "mySecretKey";

    public String gerarToken(String username) {
        return Jwts.builder()
            .setSubject(username)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + 86400000))
            .signWith(SignatureAlgorithm.HS256, SECRET_KEY)
            .compact();
    }
}
```


Autenticação com JWT (JSON Web Token)

Programação Web II

O que é uma Secret Key e como gerá-la?

A **Secret Key (Chave Secreta)** é utilizada para assinar e verificar a integridade de tokens, como o JWT.

Dessa forma, garante-se que quaisquer partes que consumam o token possam verificar sua autenticidade e integridade sem que as informações possam ser alteradas por terceiros.

A escolha de uma chave secreta forte é essencial para proteger sua aplicação, pois ataques como a força bruta podem comprometer chaves fracas ou previsíveis.

Autenticação com JWT (JSON Web Token)

Programação Web II

Como gerar uma chave secreta?

Existem diversas formas de gerar uma chave secreta segura.

Abaixo estão algumas opções:

1. Gerar manualmente (exemplo de chave segura e aleatória):

- Use uma sequência longa e difícil de adivinhar, composta por caracteres alfanuméricos e símbolos.
- Exemplos:
 - e7fGh3\$%jK29V!Xl1p0#MfQz0A

Autenticação com JWT (JSON Web Token)

Programação Web II

Como gerar uma chave secreta?

Existem diversas formas de gerar uma chave secreta segura.

Abaixo estão algumas opções:

2. Usando uma ferramenta ou código:

- No terminal usando **openssl**:

```
openssl rand -base64 64
```

Esse comando gera uma chave aleatória codificada em Base64 com 64 caracteres.

Autenticação com JWT (JSON Web Token)

Programação Web II

Como gerar uma chave secreta?

Ou em Java:

```
import java.security.SecureRandom;
import java.util.Base64;

public class SecretKeyGenerator {
    public static void main(String[] args) {
        byte[] keyBytes = new byte[32]; // 256 bits
        SecureRandom secureRandom = new SecureRandom();
        secureRandom.nextBytes(keyBytes);
        String secretKey = Base64.getEncoder().encodeToString(keyBytes);
        System.out.println("Chave Secreta: " + secretKey);
    }
}
```

É importante armazenar a chave secreta em um local seguro, como variáveis de ambiente ou um gerenciador de segredos (como *HashiCorp Vault* ou *AWS Secrets Manager*), para evitar vazamento.

Algoritmos de assinatura e o SHA-256

Programação Web II

Uma parte fundamental do JWT é o algoritmo de assinatura, utilizado para garantir a autenticidade e veracidade do token.

Ele gera a assinatura a partir da combinação do **Header**, **Payload** e da **Secret Key**, protegendo o token contra modificações.

Os algoritmos de assinatura podem ser divididos em dois tipos principais:

1. Algoritmos simétricos (HMAC):

- Usam uma única chave secreta compartilhada para assinatura e validação.
- **Exemplo:** *HS256* (HMAC com SHA-256).

2. Algoritmos assimétricos (RSA ou ECDSA):

- Usam uma chave privada para assinatura e uma chave pública para validação.
- **Exemplos:** *RS256* (RSA com SHA-256), *ES256* (ECDSA com SHA-256).

O que é SHA-256?

O **SHA-256** (*Secure Hash Algorithm 256 bits*) é um algoritmo de criptografia da família **SHA-2**, amplamente utilizado para gerar hashes criptográficos seguros.

Ele transforma uma entrada de dados em um hash de 256 bits (64 caracteres hexadecimais), garantindo que qualquer alteração na entrada produza um hash completamente diferente.

No caso do JWT, HS256 combina o SHA-256 com o algoritmo HMAC para criar um hash que depende da Secret Key, além do Header e Payload.

Isso torna o JWT confiável e seguro.

Algoritmos de assinatura e o SHA-256

Programação Web II

Exemplo de assinatura com SHA-256 (HS256):

```
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;

import java.util.Date;

public class JwtUtil {

    private static final String SECRET_KEY = "mySecretKey";

    public String gerarToken(String username) {
        return Jwts.builder()
            .setSubject(username)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + 86400000)) // 1 dia
            .signWith(SignatureAlgorithm.HS256, SECRET_KEY) // Uso do SHA-256 com HMAC
            .compact();
    }
}
```

Neste exemplo, o JWT é assinado usando o algoritmo HS256, que combina o poder do HMAC com a segurança do SHA-256.

Algoritmos de assinatura e o SHA-256

Programação Web II

Por que SHA-256 é amplamente utilizado?

- Alta segurança contra colisões (dificuldade em gerar dois hashes iguais).
- Compatível com diversos cenários – desde checagem de integridade a autenticação.
- Rápido e eficiente em várias plataformas.

Ao escolher um algoritmo de assinatura, considere o nível de segurança necessário para sua aplicação e os requisitos de desempenho.

Algoritmos de assinatura e o SHA-256

Programação Web II

Renovando

o

token

JWT

É recomendável que os tokens JWT tenham um tempo de expiração curto e sejam renovados antes de seu vencimento para manter a segurança da aplicação.

A renovação pode ser feita emitindo um novo token baseado no token original, verificando sua validade e autenticidade.

Algoritmos de assinatura e o SHA-256

Programação Web II

Renovando

o

token

JWT

Exemplo de método para renovar o token:

```
public String renovarToken(String tokenAntigo) {  
    String username = Jwts.parser()  
        .setSigningKey(SECRET_KEY)  
        .parseClaimsJws(tokenAntigo)  
        .getBody()  
        .getSubject();  
  
    return Jwts.builder()  
        .setSubject(username)  
        .setIssuedAt(new Date())  
        .setExpiration(new Date(System.currentTimeMillis() + 86400000))  
        .signWith(SignatureAlgorithm.HS256, SECRET_KEY)  
        .compact();  
}
```

No exemplo acima, o método `renovarToken` verifica o token antigo, extrai o nome de usuário do *claim* e emite um novo token com um novo tempo de expiração.

Incluindo permissões do usuário nas claims

Adicionar permissões ou papéis (**roles**) do usuário nas *claims* do JWT ajuda a incluir dados contextuais no token e elimina a necessidade de consultas adicionais enquanto o token é processado.

Exemplo de geração de um token com permissões:

```
public String gerarTokenComPermissoes(String username, List<String> permissoes) {  
    return Jwts.builder()  
        .setSubject(username)  
        .claim("roles", permissoes)  
        .setIssuedAt(new Date())  
        .setExpiration(new Date(System.currentTimeMillis() + 86400000))  
        .signWith(SignatureAlgorithm.HS256, SECRET_KEY)  
        .compact();  
}
```


Exemplo de validação e extração das permissões:

```
public List<String> getPermissoesDoToken(String token) {  
    Claims claims = Jwts.parser()  
        .setSigningKey(SECRET_KEY)  
        .parseClaimsJws(token)  
        .getBody();  
  
    return claims.get("roles", List.class);  
}
```

Com isso, ao gerar o token, você pode passar as permissões relevantes do usuário (como ["ROLE_ADMIN", "ROLE_USER"]) para serem incluídas como parte do token.

E, ao validar o token, você pode facilmente extrair essas permissões. Assim, as permissões do usuário ficam disponíveis em todas as requisições onde o token JWT está presente.

O que é OAuth2?

OAuth2 é um protocolo de autorização que permite o acesso seguro a APIs de terceiros sem compartilhar as credenciais do usuário.

Fluxo de autorização (Authorization Code Flow)

1. **Solicitação de autorização:** a aplicação solicita permissão ao usuário para acessar seus dados.
2. **Concessão de código de autorização:** o usuário consente e recebe um código temporário.
3. **Troca por token de acesso:** a aplicação troca o código por um token de acesso.
4. **Acesso aos recursos protegidos:** o token é usado para acessar os recursos da API.

Autenticação OAuth2 com GitHub e Google

A integração de autenticação **OAuth2** com provedores como *GitHub* e *Google* permite que você implemente **SSO (Single Sign-On)** em sua aplicação Spring Boot.

Nesse caso, o Spring Security cuida de grande parte da configuração, possibilitando que sua aplicação autentique usuários por meio de provedores OAuth2 de maneira simplificada.

Leia mais: [Spring Boot OAuth2](#)

Introdução ao SSO e OAuth2

Programação Web II

Configurações:

pom.xml

```
<dependency>  
  <groupId>org.springframework.boot</groupId>  
  <artifactId>spring-boot-starter-oauth2-client</artifactId>  
</dependency>
```

build.gradle

```
dependencies {  
    implementation  
    'org.springframework.boot:spring-boot-starter-oauth2-client'  
}
```

Introdução ao SSO e OAuth2

Programação Web II

Configuração no `application.yml`

```
spring:
  security:
    oauth2:
      client:
        registration:
          github:
            client-id: <SEU_CLIENT_ID_GITHUB>
            client-secret: <SEU_CLIENT_SECRET_GITHUB>
            scope: read:user
            redirect-uri: "{baseUrl}/login/oauth2/code/github"
            client-name: GitHub
          google:
            client-id: <SEU_CLIENT_ID_GOOGLE>
            client-secret: <SEU_CLIENT_SECRET_GOOGLE>
            scope: openid,profile,email
            redirect-uri: "{baseUrl}/login/oauth2/code/google"
            client-name: Google
        provider:
          github:
            authorization-uri: https://github.com/login/oauth/authorize
            token-uri: https://github.com/login/oauth/access_token
            user-info-uri: https://api.github.com/user
          google:
            authorization-uri: https://accounts.google.com/o/oauth2/v2/auth
            token-uri: https://oauth2.googleapis.com/token
            user-info-uri: https://www.googleapis.com/oauth2/v3/userinfo
```

Configuração de redirecionamento padrão:

Ao usar OAuth2, o Spring Security, por padrão, redireciona o fluxo de autenticação para o **endpoint /login**.

Se você deseja alterar o comportamento de redirecionamento ou proteger recursos específicos, pode configurar isso na classe **SecurityConfig**.

Configuração da segurança

A seguir está um exemplo de configuração personalizada para redirecionamento de início de sessão e proteção de endpoints:

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(authorize -> authorize
                .requestMatchers("/", "/publico", "/login").permitAll()
                .anyRequest().authenticated()
            )
            .oauth2Login(oauth2 -> oauth2
                .defaultSuccessUrl("/usuario", true) // Redireciona após login bem-sucedido
                .loginPage("/login") // Página de login personalizada (opcional)
            );

        return http.build();
    }
}
```

Obtendo informações do usuário autenticado

Depois que um usuário realiza login com um provedor OAuth2, as informações do perfil do usuário podem ser recuperadas de um objeto **OAuth2AuthenticationToken**.

```
@RestController  
  
public class UsuarioController {  
  
    @GetMapping("/usuario")  
    public Map<String, Object> obterUsuario(@AuthenticationPrincipal OAuth2User principal) {  
        return principal.getAttributes(); // Retorna os atributos do usuário autenticado  
    }  
}
```

Obtendo informações do usuário autenticado

Exemplo de atributos retornados ao usar OAuth2:

- **GitHub:** os atributos incluem o login, a URL do avatar, o ID do usuário, etc.
- **Google:** os atributos incluem o e-mail, nome, foto de perfil, etc.

Esta configuração permite que você integre autenticação OAuth2 com os provedores **GitHub** e **Google** sem a necessidade de lidar diretamente com os tokens *OAuth2*.

O Spring Security cuida da maior parte das etapas, oferecendo uma configuração robusta e segura. Certifique-se de configurar corretamente os IDs do cliente e as chaves secretas fornecidos pelos provedores OAuth2 para que a integração funcione corretamente.

Exercícios

Exercício 1: Configurar autenticação JWT na aplicação

Escolha um projeto já desenvolvido em aula e complemente com:

- Configurar autenticação JWT na aplicação Spring Boot para proteger endpoints com base em tokens.
- Integrar a aplicação com um provedor de SSO fictício (ex.: Keycloak, Okta) utilizando OAuth2. (Desafio)
- Testar o fluxo completo de geração, troca e verificação de tokens, garantindo que apenas usuários autenticados tenham acesso aos recursos protegidos.

Obrigada